AI Coding Agents: A Practical Reading Guide

This guide brings together eight sets of resources for students using AI coding agents in software and machine learning projects. It answers three practical questions: what should I ask the agent to do, how should I work with it, and how do I check the result?

1. MIT Missing Semester: understand what an agent does

The 2026 Agentic Coding lecture is a useful introduction. A coding agent is a conversational model connected to tools that can inspect files, edit code, and run commands. Unlike a text only chat, it can act on a project and use the results of those actions to continue working. [1]

The lecture demonstrates this with a short Python script: the agent turns it into a command-line program, adds type annotations, and checks them. The teaching point is the repeated cycle of making a change and checking it, rather than a single impressive answer. Other uses include debugging, refactoring, code review, and exploring unfamiliar projects. [1]

It explains why agents need relevant context: the information available during a task. Instruction files, reusable workflows, smaller helper agents, and outside tool connections help organize that work. Exercises compare manual coding, autocomplete, chat, and agents. The cautions matter: review correctness and security, check what reaches cloud services, and do not confuse autonomy with reliability. [1]

2. OpenAI Codex: give the task a clear shape

OpenAI’s best practices guide recommends four parts to a task: the goal, relevant context, constraints, and completion criteria. In practical terms, explain the requested change, point to the files or errors that matter, state what must stay unchanged, and describe how success will be checked. For ambiguous or larger changes, agree on a plan before implementation. [2]

The guide also covers configuring the working environment, reviewing changes, connecting outside tools, turning repeated work into skills, and automating established workflows. These are later steps, not prerequisites for a first exercise. Start with a task whose result you can inspect. [2]

Keep recurring project instructions in AGENTS.md

AGENTS.md is a project instruction file that Codex reads before working. Use it for information that should apply across tasks: how to run the project, which tests to run, conventions to follow, and files or operations to avoid. This saves students from repeating the same instructions in every conversation. [2, 3]

Codex combines guidance from different locations. More local instructions can override broader ones, and the combined text has a size limit. Keep the file concise and check which instructions were loaded when behavior is unexpected. Treat it as guidance, not as a substitute for permissions or code review. [3]

3. Claude Code: require evidence, not a claim of success

Anthropic's main recommendation is to give the agent something it can run to check its work: a test, a build, a comparison script, or a visual check. A response saying that a bug is fixed is not the same as showing the command, result, and corrected behavior. [4]

For larger changes, the guide separates exploration, planning, implementation, and committing. Planning is useful when the code is unfamiliar or several files will change; it is unnecessary overhead for every typo. Prompts should identify the relevant files, existing patterns, and expected behavior. [4]

The guide also recommends short CLAUDE.md instructions, appropriate permissions, early corrections, and fresh sessions for unrelated tasks. Skills, hooks, subagents, plugins, and parallel sessions support more advanced workflows. Do not let repeated unsuccessful fixes turn into an unexamined chain of changes. [4]

What the quickstart adds

The quickstart covers the first practical steps: install the tool, sign in, open a project, ask about its structure, make a small change, and use Git and tests. It assumes a terminal, a project, and supported account access. Follow the current installation instructions for the relevant operating system rather than copying an old command from a slide. [5]

4. Cursor Learn: follow the development process

Cursor Learn provides a guided sequence rather than one long reference manual. Its foundations cover model limitations, tokens and pricing, context, tool use, and agents. The coding lessons then move through understanding a project, developing features, debugging, reviewing, testing, and customization. It teaches programming with AI tools, not how to train machine learning models. [6]

Understand the existing code. Ask where a behavior is implemented and how information moves through the application. Search precisely when you know a function name; ask broader questions when exploring. Check for an existing helper before creating another one. [7]

Build in checkable steps. The feature development lesson recommends a plan for substantial work and tests with clear expected outcomes. In a test first exercise, review the tests, confirm that the new behavior is missing, and then implement it without quietly weakening the checks. [8]

Debug from evidence. Record the inputs and steps that trigger a bug, reduce the example, and change one thing at a time. Logs and a specific explanation of the suspected cause are more useful than repeated requests to "try again." [9]

Review the result. Passing tests do not rule out missed cases, security problems, or duplicated logic. Read the changes and keep them understandable for another reviewer. A diff is the comparison showing what was added, removed, or changed. [10]

Customize only where useful. Rules hold recurring conventions; skills package specialized workflows; MCP (Model Context Protocol) connects agents to outside tools and information. Begin with a small set of project-specific instructions rather than adding every available integration. [11]

5. GitHub Skills: learn by changing a working project

The GitHub Copilot exercise uses an extracurricular activities website for a fictional high school. Students work from an existing codebase instead of facing an empty folder. The exercise uses a preconfigured Codespace with VS Code in the browser, introduces ways to explain, plan, and develop code, and includes pull request summarizing and review. [12]

Its listed prerequisites include the Introduction to GitHub exercise, familiarity with VS Code, and basic coding principles. The repository is therefore a useful starting point for students who already understand elementary programming, not a replacement for teaching those fundamentals. [12]

6. Stanford CS146S: a reference for larger course design

The Modern Software Developer examines how people and agents work together across a software project. The course covers specifying requirements, supplying context and tools, building in repeated cycles, and evaluating results. It combines lectures, hands on work, practitioner talks, and a final project. [13]

The stated prerequisite is programming experience equivalent to CS111/CS161, with CS221/229 recommended. This makes it a better reference for designing advanced activities than a complete first assignment for beginners. [13]

7. scikit-learn: check the experiment as well as the code

Data leakage and consistent preprocessing

Data leakage means that model development uses information that would not legitimately be available for a new prediction. Split the data before fitting preprocessing or selecting features. Learn those transformations from training data only, then apply the same learned transformations to validation or test data. [15]

A Pipeline helps keep preprocessing and prediction together. During cross validation, put learned preprocessing inside the pipeline so each training fold fits its own transformations. A pipeline does not repair an unsuitable data split. Also record random state choices: controlling randomness helps repeat an experiment, but repeating one split does not establish robustness. [15]

Probability calibration

Calibration asks whether probabilities match observed frequencies. For example, among cases assigned a positive class probability near 0.70, approximately 70% should belong to that class. This is a statement about groups of predictions, not a guarantee for an individual case or the model's overall accuracy. [16]

Use a calibration curve to compare predicted probabilities with observed frequencies. Fit calibration using held out or out of fold predictions and keep final evaluation data separate. The documentation discusses sigmoid, isotonic, and temperature based methods. Brier score and log loss measure more than calibration alone, so neither should be presented as a pure calibration score. [16]

8. Playwright: test what the user can actually do

Playwright's testing guide emphasizes visible behavior rather than internal implementation details. A browser test should follow the interaction a person performs and check the resulting page or output. Checking that a function was called is not enough to establish that a user completed the task. [17]

The guide recommends independent tests, controlled data, and stable ways of finding elements, such as their role or label. Use assertions that wait for the expected state rather than relying on arbitrary delays. Avoid making tests depend on outside services you do not control, and use debugging tools and traces to investigate failures. [17]

9. A workflow students can reuse

The following workflow combines the readings into a small, repeatable exercise. Use it on a project that already runs.

Begin with the project, not the feature request

Run the application and its existing checks before editing. Ask the agent to locate the code involved and explain the current behavior. Keep a recoverable starting version. This makes it easier to separate an existing problem from one introduced by the new change. [7, 10]

Write a task with an observable finish

A request such as “improve the CSV tool” leaves important decisions unstated. The following original example applies the task structure in the Codex guide. [2]

Add validation to the CSV upload in this project. First locate the upload handler and existing tests.

Reject files with missing required headers and show which headers are missing. Leave the original file unchanged. Do not add a new dependency.

The change is complete when a valid file still loads, a file with a missing header shows the correct error, and existing tests pass. Add tests for these cases and report exactly what you ran.

Make the check meaningful

Review the expected outcomes before implementation. For a bug fix, reproduce the failure; for a new feature, verify that the test detects the missing behavior. After the change, repeat that check and run the relevant existing tests. A failure caused by a broken test setup is not evidence that the intended requirement was tested. [8, 9]

Save only the rules that apply to every task

For a Python project that already uses pytest, this is a possible starting point for AGENTS.md. Adjust it to the actual repository; Claude Code uses CLAUDE.md for its project guidance. [3, 4]

```
# Project rules
Keep the original input files unchanged.
Use the dependencies already listed in the project.
Run relevant tests with python -m pytest.
Do not remove tests to make a change pass.
Explain any proposed change to an expected result.
Report changed files, checks run, and checks not run.
```

Inspect the change and report its limits

Read the diff and try the feature yourself. Ask about unexplained edits, removed tests, extra dependencies, and error handling. Keep the final change focused. If a check could not run, record the reason instead of treating it as a pass. [10]

References

[1] MIT Missing Semester (2026). Agentic Coding.
https://missing.csail.mit.edu/2026/agentic-coding/

[2] OpenAI. Best practices. Codex / ChatGPT Learn.
https://developers.openai.com/codex/learn/best-practices

[3] OpenAI. Custom instructions with AGENTS.md. Codex / ChatGPT Learn.
https://developers.openai.com/codex/guides/agents-md

[4] Anthropic. Best practices for Claude Code. Claude Code Docs.
https://code.claude.com/docs/en/best-practices

[5] Anthropic. Quickstart. Claude Code Docs.
https://code.claude.com/docs/en/quickstart

[6] Cursor. Cursor Learn. Course overview.
https://cursor.com/learn

[7] Cursor. Understanding your codebase. Cursor Learn.
https://cursor.com/learn/understanding-your-codebase

[8] Cursor. Developing features. Cursor Learn.
https://cursor.com/learn/creating-features

[9] Cursor. Finding and fixing bugs. Cursor Learn.
https://cursor.com/learn/finding-fixing-bugs

[10] Cursor. Reviewing and testing code. Cursor Learn.
https://cursor.com/learn/reviewing-testing

[11] Cursor. Customizing agents. Cursor Learn.
https://cursor.com/learn/customizing-agents

[12] GitHub Skills. Getting started with GitHub Copilot. Exercise repository.
https://github.com/skills/getting-started-with-github-copilot

[13] CS146S. The Modern Software Developer. Stanford course website.
https://themodernsoftware.dev/

[14] Eric, M. Assignments for CS146S: The Modern Software Developer. Repository and setup notes.
https://github.com/mihail911/modern-software-dev-assignments

[15] scikit-learn developers. Common pitfalls and recommended practices. User Guide. https://scikit-learn.org/stable/common_pitfalls.html

[16] scikit-learn developers. Probability calibration. User Guide. https://scikit-learn.org/stable/modules/calibration.html

[17] Playwright. Best Practices. Official documentation. https://playwright.dev/docs/best-practices